OPENCLASSROOMS

Mettre en forme des composants React avec `styled-components`

Parcours - [Développeur d'application React]
> Projet 4 - Développez des sites complexes avec React

Cette fiche pratique va vous permettre d'appliquer du style à des composants react en utilisant la bibliothèque `styled-components`.

Pour simplifier l'écriture nous prendrons le contexte d'une application React Simple.

Si vous êtes sur Next.js (App Router), évitez de mettre `"use client"` partout : isolez l'usage de `styled-components` (via quelques composants dédiés, un Provider ou un registry) dans des ***Client Components***, et gardez le reste de l'UI en ***Server Components*** dès que possible.

Recommandé si	À éviter si
• Votre UI est majoritairement composée de **Client Components** • Vous avez besoin de **style dynamique** (props, state, conditions) • Vous cherchez une **API ergonomique** : CSS dans JS, thèmes, variables • Vous travaillez dans une application React "classique" (CRA, Vite, Astro framework mode client)	• Vous construisez une app **full Server Components** (ex : architecture App Router Next.js 100 % RSC) • Vous souhaitez un CSS **100 % statique** (préférer vanilla-extract, Tailwind ou CSS Modules) • Vous générez des styles sensibles aux states côté serveur

Vous pourrez retrouvez le code de cette fiche sur Stackblitz.

Installer la librairie `styled-components`

```bash
yarn add styled-components
```

Créer un Styled Component

```jsx
import styled from "styled-components"
```

Dans Next.js (App Router), styled-components doit être utilisé dans un Client Component. Ajoutez `"use client"` en haut du fichier (avant les imports) pour les composants UI qui utilisent styled-components.

Voici un exemple de composant Card :

```
“use client” //si vous utilisez Next.js

import styled from 'styled-components';

const CardLabel = styled.span`
 color: #5843e4;
 font-size: 22px;
 font-weight: bold;
`;

const CardWrapper = styled.div`
 display: flex;
 flex-direction: column;
 padding: 15px;
 background-color: #eee;
 border-radius: 30px;
 width: 350px;
 transition: 200ms;
 &:hover {
   cursor: pointer;
   box-shadow: 2px 2px 10px #e2e3e9;
 }
`;
```

```
function Card({ label, title }) {
 return (
   <CardWrapper>
     <CardLabel>{label}</CardLabel>
     <span>{title}</span>
   </CardWrapper>
 );
}

export default Card;
```

Utilisation :

```jsx
<Card label="Card" title="Mon premier StyledComponent" />
```

On peut voir dans cet exemple que mon composant Card est composé de 2 styled components CardWrapper et CardLabel. L'un des principaux avantages des StyledComponents c'est que nous pouvons facilement retoucher l'aspect des composants directement dans le composant et non dans un fichier à part.

Styliser un composant React ou issu d'une librairie

Pour styliser un composant issu d'une librairie il faut passer le composant entre parenthèses de `styled` et comme pour nos composants précédents, ajouter du style.

```
import styled from 'styled-components';
import { Link } from 'react-router-dom';

 //.....code
const StyledLink = styled(Link)`
padding: 12px 16px;
color: #8186a0;
text-decoration: none;
font-size: 18px;
`;

function Card({ label, title }) {
 return (
   <CardWrapper>
```

```
      <CardLabel>{label}</CardLabel>
      <span>{title}</span>
      <StyledLink to="https://openclassrooms.com">OpenClassrooms</StyledLink>
    </CardWrapper>
  );
}

export default Card;
```

Dans notre exemple ici, nous avons formaté un composant `Link` de `react-router-dom`.

Générer du style dynamique avec des props

Il est possible d'ajouter du style dynamique pour pouvoir ajuster notre composant en fonction des props qu'ils reçoit.

Faisons maintenant en sorte que notre composant StyledLink change d'aspect lorsque qu'il reçoit la props $primary :

```jsx
const StyledLink = styled(Link)`
  color: #8186a0;

  ${(props) =>
    props.$primary &&
    `
    color: white;
    background: #5843e4;
    border-radius: 30px;
  `}
`
```

Le préfixe $ est toujours recommandé pour éviter que la prop n'apparaisse dans le DOM.

Utiliser un thème et des variables

Styled components peut aider à standardiser le style d'une application notamment grâce au `ThemeProvider` qui permet d'avoir accès à des variables dans tout le code.

colors.js

```jsx
const colors = {
  primary: "#5843E4",
  secondary: "#8186A0",
  backgroundLight: "#EEE",
}

export default colors
```

Provider (client)

```jsx
import { ThemeProvider } from "styled-components"
import colors from "./colors"

export default function AppLayout({ children }) {
  return <ThemeProvider theme={{ colors }}>{children}</ThemeProvider>
}
```

Utilisation

```
const CardWrapper = styled.div`
 display: flex;
 flex-direction: column;
 gap: 10px;
 padding: 15px;
 background-color: ${({ theme }) => theme.colors.backgroundLight};
 border-radius: 30px;
 width: 350px;
 transition: 200ms;
 &:hover {
   cursor: pointer;
   box-shadow: 2px 2px 10px #e2e3e9;
 }
`;
```

Pseudo-selectors & autres syntaxe CSS

Comme dans du CSS classique nous avons parfois besoin d'utiliser des pseudo-selector. Comment nous gérer cela ? Grâce au symbole `&` comme dans l'exemple ci dessous :

```
jsx

const CardWrapper = styled.div`
  transition: 200ms;

  &:hover {
    cursor: pointer;
    box-shadow: 2px 2px 10px #e2e3e9;
  }
`
```

Créer un style global

Nous pouvons également créer un style global que nous pourrons appliquer à notre application grâce à `createGlobalStyle` que nous pouvons utiliser comme ci-dessous.

```
jsx


import { createGlobalStyle } from "styled-components"

const GlobalStyle = createGlobalStyle`
  body {
    font-family: system-ui, sans-serif;
    background: #aaabc8;
  }
`


export default GlobalStyle
```

Utilisation

Nous pouvons ensuite ajouter notre composant à la racine de notre application

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App.jsx';
```

```
import AppLayout from './layout/AppLayout.jsx';
import GlobalStyle from './layout/GlobalStyle';

createRoot(document.getElementById('root')).render(
 <StrictMode>
   <AppLayout>
     <GlobalStyle />
     <App />
   </AppLayout>
 </StrictMode>
);
```

Nous pouvons également utiliser ici les variables du thème dans notre GlobalStyle car il est contenu à l'intérieur de `AppLayout`

À retenir

À éviter :

- Utiliser **styled-components** dans un **Server Component** (ou mélanger Server/Client dans le même fichier).
- Générer des styles dynamiques à partir de données **uniquement connues côté serveur** (calculez côté serveur, passez une valeur simple au client).
- Oublier les **transient props** : sans préfixe $, vos props de style peuvent se retrouver dans le DOM.

Toujours :

- Ajouter `"use client"` dès que vous déclarez des styled components.
- Centraliser vos variables (couleurs, spacing, radius) via `ThemeProvider` pour garder un style cohérent.
- Garder une séparation claire : **Server Components = données/logique, Client Components = UI + styles + interactions.**